

Module 1 — Complete Evaluator Q&A + Talking Script

Everything you need for the 11 standard evaluation questions, specific to Module 1 (Dynamic Scenario Generation). Every number and file reference below is real, pulled directly from the code, the evaluation results, and the literature review — nothing invented. Last compiled 2026-08-11.

1. Theoretical specifics related to the domain

Simple version: Module 1 automatically builds a training mission — the rooms, the doors, where the trainee/hostage/terrorists stand, what patrol routes look like, what furniture is where — from a small set of settings an evaluator picks (room count, difficulty, etc). It does this using a technique called "**constructive generation with constraint satisfaction**" — meaning it *builds* the layout piece by piece according to rules, then *checks* the result is valid, rather than randomly guessing until something works.

The theory behind it, in order:

- The room layout is a **graph** (rooms = nodes, doors = edges), built as one of four topology shapes — linear (a chain), branching (a tree), hub-and-spoke (a star), or loop (a ring). This is explicit in ``LayoutGenerator.cs`` — see ``GenerateLinearTopology``, ``GenerateBranchingTopology``, ``GenerateHubAndSpokeTopology``, ``GenerateLoopTopology``.
- We deliberately did **not** use Wave Function Collapse (WFC), a very popular procedural-generation technique, because a paper by **Karth & Smith (2017, "WaveFunctionCollapse is Constraint Solving in the Wild")** proved WFC "cannot enforce global reachability without excessive backtracking" — meaning WFC can produce rooms you literally can't walk between, and fixing that gets expensive. Since a training mission that can't be completed is worse than useless, we built our own constructive graph generator instead, and check reachability as a separate, explicit final step (``ValidateReachability``, ``LayoutGenerator.cs:839``).
- Doors are modeled as **attributes of rooms**, not as their own separate graph objects — this follows **Smith & Whitehead (2010, "Analyzing the Expressive Range of a Level Generator")**, which found that treating doors as independent design elements makes connectivity much harder to guarantee.
- NPC roles (who patrols, who guards, who's stationary) are assigned using **BFS depth** — literally "how many rooms deep is this NPC from the entrance" — which follows **Space Syntax theory** (Dahl & Rinde, 2008 thesis): rooms deep in the graph are treated as private/guarded, rooms near the entrance are public/patrolled. This is not random — it's spatially derived, on purpose (see Q2, novelty).
- Entity placement (where hostages/terrorists actually stand) follows a **hierarchical three-stage placement order** — big/important things first (trainee, then hostage, then terrorists) — from **Raistrick et al. 2024, "Infinigen Indoors" (CVPR)**, a real-world large-scale procedural indoor generation system.
- The "danger level" of where terrorists get placed (out in the open vs. behind

doors vs. blind corners) is grounded in a **real VR police-training stress study** — Zechner et al. 2023 ("Enhancing Operational Police Training in High Stress Situations with VR") — which measured that a stranger suddenly entering a room was the single highest-stress, highest-anxiety event they tested (mean stress 41.45/50). Higher difficulty settings place terrorists in positions designed to recreate that specific stressor.

SAY THIS:

"Module 1 is grounded in procedural-content-generation research, not improvised. We specifically rejected Wave Function Collapse — a well-known generation technique — because a 2017 formal analysis by Karth and Smith proved it can't guarantee a level is fully walkable without expensive backtracking, and an unplayable training mission isn't acceptable. Instead we built a constructive graph generator with an explicit, separate reachability check, doors modeled as room attributes rather than independent objects (following Smith & Whitehead 2010), and NPC roles derived from Space Syntax — how spatially deep a room is from the entrance — rather than assigned randomly."

2. Novelty

Simple version: Nobody else has built a system that does *all five* of these things together in one pipeline. Plenty of tools do one or two.

The precise claim, straight from the literature review's gap analysis:

"No existing system in the reviewed literature combines all of the following capabilities within a single integrated pipeline: (a) parameter-driven generation of indoor layouts, (b) entity placement with constraint-based spatial validation, (c) NPC role assignment and role-based navigation context generation, (d) scenario validation, and (e) output of a structured scenario artifact (Scenario JSON) that can be consumed by runtime NPC behaviour, cognitive analysis, and post-mission review modules within one unified system."

Broken into plain English: existing scenario-generation research (military training scenario generators like Zook et al. 2012, Niehaus et al. 2011) builds *event sequences* (what happens, in what order) but not the *physical space* those events happen in — the paper that inspired ours (Zook et al. 2012) even says spatial generation is "a separate, complementary problem," i.e. explicitly not solved by that line of research. Meanwhile, indoor-space-generation research (ProcTHOR, Infinigen Indoors) builds realistic rooms and furniture, but has nothing to do with military/tactical training — no NPC roles, no mission validation, no downstream integration with an AI behaviour system or an after-action-review dashboard. **Module 1 sits exactly in the gap between those two bodies of research** and is the first system (per the sources we could find) to combine them for VR tactical training specifically.

SAY THIS:

"The novelty isn't any single technique — every individual piece has academic precedent. The novelty is the combination: nothing in the literature we found generates a full, validated, role-assigned indoor training mission that also plugs directly into a downstream NPC-behaviour system and an after-action review dashboard, in one pipeline, from evaluator-set parameters. Scenario generation research does events, not space. Indoor generation research does space, not tactical training roles or mission validation. We do both, connected."

3. Result

Simple version: We ran five separate experiments on real generated data (not simulated/estimated) to answer four core questions. Headline results:

Experiment	Question	Headline result
A / A2 — Variability	Does repeating the same settings produce different missions?	Room <i>topology</i> is identical every time (0% variation) at fixed settings — but placement, roles, doors, and furniture vary a lot (e.g. avg. distance between two generated entity layouts = 10.06m). Turning "randomness" up to High barely changes this.
B — Traceability	Does changing a setting actually change the right thing in the output?	Yes, overwhelmingly — every one of 15 tested parameter→metric relationships was statistically significant at $p < 0.0001$ with a "large" effect size, except one: the "multiple entries" setting only affects entry-point count, not interior connectivity (a known, documented limitation).
C — Randomness sensitivity	Does the "randomness" dial actually do anything?	Mostly no — room topology stayed completely constant across low/medium/high randomness. Only one minor metric drifted, and not significantly.
D — Difficulty	Does difficulty level actually change the mission, not just cosmetically?	Yes — at max difficulty, patrol routes disappear entirely (patrol count and coverage both drop to 0), and the minimum hostage-to-terrorist distance rises steadily from 3.00m → 3.51m → 3.93m as difficulty increases.
E — Validation reliability	How often does a generated mission actually pass all quality checks?	100% pass rate (43/43) within the range evaluators can actually select in the current form (3–5 rooms, 1–4 terrorists). Across the full technical range the pipeline supports (up to 15 rooms, 8 terrorists), pass rate drops to 83.2% overall — the failures are concentrated entirely outside what evaluators can currently select.

Full numbers live in

`Assets/Module1\_DataModels\_and\_IO/Output/EvaluationResults/MODULE1\_RESULTS.md` and are mirrored live on the dashboard (`sentinels-aar`, Module 1 tab).

SAY THIS:

"Across five experiments and roughly 4,000 generated scenarios total, we found three honest things: first, every parameter an evaluator can set genuinely and measurably changes the right part of the output — that's not automatic in procedural generation, and we tested it rather than assumed it. Second, within the range evaluators can actually select, one hundred percent of generated missions pass every validation check — zero broken scenarios. Third, we found and reported a real limitation honestly: the 'randomness' dial barely affects anything at the current settings, because room topology is deterministic by design at this scale — that's documented as a finding, not hidden."

4. How did we evaluate? How did we decide the criteria and methodology?

Simple version: We didn't invent "evaluation criteria" out of nowhere — we wrote down four research questions *first* (before writing evaluation code), derived directly from the literature gap, and then built one experiment per question.

The four research questions (RQ1–RQ4), straight from the literature review:

- 1 **RQ1** — Can we generate layouts that are spatially valid (reachable, non-overlapping, structurally sound)? → tested by **Experiment E**.
- 1 **RQ2** — Can NPC roles/navigation be derived from spatial structure rather than assigned arbitrarily? → tested by **Experiment D** (does difficulty meaningfully reconfigure roles) and reflected in the role-assignment design itself (BFS-depth-based, not random).
- 1 **RQ3** — Does the pipeline reliably produce loadable, mission-ready scenarios? → tested by **Experiment E** (validation pass rates).
- 1 **RQ4** — Does the generator produce measurable variability across repeated runs with identical parameters, and do parameters have traceable, predictable effects? → tested by **Experiment A/A2** (variability) and **Experiment B** (traceability) and **Experiment C** (randomness control).

Method, concretely:

- ``BatchEvaluationRunner.cs`` runs each experiment (one ``[ContextMenu]`` action per experiment inside Unity) and generates the scenarios.
- ``ScenarioMetrics.cs`` extracts **39 numeric measurements** from every generated scenario — room-graph shape, door states, entity spread, navigation complexity, furniture density — so every scenario becomes one comparable row of data.
- ``DiversityAnalyser.cs`` compares scenarios *pairwise* — for Experiment A that's **4,950 unique pairs** from 100 scenarios ($C(100,2)$) — across 7 different "how different are these two missions" measures (graph edit distance, entity position distance, role distribution distance, etc).
- The actual statistics were run in Python (pandas/scipy/scikit-posthocs) on exported CSVs, outside Unity. Method: **Shapiro-Wilk** test first to check if data is normally distributed; if yes, **ANOVA + Tukey HSD**; if no (which was true almost everywhere), **Kruskal-Wallis + Dunn's test with Bonferroni correction**. Effect sizes: η^2 (**eta-squared**) for ANOVA, ϵ^2 (**epsilon-squared**) for Kruskal-Wallis, with standard thresholds (negligible <0.01 , small ≥ 0.01 , medium ≥ 0.06 , large ≥ 0.14).
- For comparing "medium randomness" against "high randomness" diversity distributions specifically (Experiment A vs A2), we used **Mann-Whitney U** with **rank-biserial correlation** as the effect size.

Why these specific stats, simply: Kruskal-Wallis instead of plain ANOVA because our data mostly wasn't normally distributed (we checked with Shapiro-Wilk rather than assuming), and Kruskal-Wallis doesn't require that assumption. Bonferroni correction on the follow-up tests because we're running many comparisons at once, and that correction keeps us from claiming a "false positive" significant result just by chance.

SAY THIS:

"We started from four research questions derived directly from the gap in the literature review, then designed one experiment per question — not the other way around. Every scenario in every experiment is measured the same way, through a 39-metric extractor, so results are comparable across experiments. We used Shapiro-Wilk to check whether our data was normally distributed before picking a statistical test — it mostly wasn't, so we used Kruskal-Wallis with Bonferroni-corrected post-hoc tests rather than assuming a parametric test was safe to use."

5. Challenges faced during research/development, and how we addressed them

Pick two or three of these for a live talk — they're real, specific, and each has a clean "problem → why it mattered → fix" shape.

A. A hidden bug that would have invalidated our own variability study.

Our seed fallback used `Environment.TickCount`, which only changes every ~15 milliseconds on Windows — but generating one scenario takes about 1 millisecond. Generating 100 "independent" scenarios in a tight loop would have silently handed many of them the *identical* seed, making our own variability experiment measure nothing (an artificially low or zero result) without ever throwing an error. **Fix:** built a dedicated `SeedSequence` class that tracks every seed it's already issued and guarantees no repeats, while staying fully reproducible from one master seed (`BatchEvaluationRunner.cs:1044-1066`).

B. A CSV corruption bug that produced no visible symptom. Our metrics exporter built each row by appending fields with `sb.Length > 0` deciding whether to add a comma separator. This happened to work only because the first column was never empty — if the column order had ever changed, an empty field partway through a row would silently swallow its own comma and shift every following column by one, producing a corrupted-but-still-readable-looking CSV with no error thrown. **Fix:** rewrote every row builder to assemble a `string[]` array first and `string.Join` it — column count is then structurally guaranteed correct no matter what the data looks like.

C. A finding that looked like a bug but was real, correct behaviour.

During an internal audit, one metric (`doorsOpen`) showed exactly zero across an entire 300-door batch — looked broken. Investigation showed it was genuinely correct: our door-state rule locks any door touching the hostage room and closes any door touching the entry room for realism, and at our specific 4-room baseline, no generated layout happens to have a door that's neither of those. We documented this explicitly as a scale-dependent limitation rather than quietly "fixing" what wasn't actually broken.

D. Three "independent" diversity metrics turned out to be mathematically the same thing — but only at small scale. Graph-edit-distance, role-distribution-distance, and depth-profile-distance were identical in 100% of 4,950 compared pairs at our 4-room baseline. We re-ran the same comparison at 12 rooms and they completely decoupled (0% pairs equal). At 4 rooms, any single change to the room graph mechanically also

moves exactly one NPC's role and depth — the metrics aren't broken, they're just genuinely coupled at that specific small scale. Documented, not hidden.

E. Cross-team integration friction. When Module 2 (NPC behaviour) started consuming Module 1's output, the two systems' code couldn't even compile together at first — Module 1's assembly definition was sealed off from ``Assembly-CSharp`` where Module 2's classes live. Fixed by relocating integration-layer scripts (``SceneBuilder.cs``) outside Module 1's assembly boundary. Beyond that, Module 2's team formally logged real, measured integration gaps (``MODULE1_REQUESTS.md``) — e.g. terrorists had a 0-hit cover system because Module 1 never emits cover points, and exactly 0 of 6 corridor nav-points were reachable by any terrorist because an ``isExterior`` door field was declared but never actually set.

SAY THIS:

"The most important challenge wasn't a bug in the generator — it was a bug in our own measurement of the generator. Our seed fallback would have silently made a 100-scenario 'variability' experiment measure almost nothing, because Windows' tick counter doesn't change fast enough between generations. We caught it by building an explicit seed-tracking system before trusting any of our own numbers. We also had at least two cases where something that LOOKED like a bug — an all-zero metric, three metrics that never disagreed — turned out to be genuinely correct behaviour at our test scale, and we verified that by testing at a different scale rather than just patching the symptom."

6. Future work — what we see coming for this module

Straight from the results' own limitations section and the continuation plan:

- ****Expose the controls we already validated but haven't shipped to evaluators.**** Difficulty, randomness, and hostage-risk-level are all proven to work (Experiments C/D) but the live evaluator form currently locks them at fixed values (difficulty=3, randomness=medium). Turning these on is mostly a UI change, not new generation work.
- ****Widen the evaluator-facing room/terrorist range to match what the engine already supports.**** The form currently allows 3–5 rooms / 1–4 terrorists; the pipeline itself handles up to 15 rooms / 8 terrorists, just with a lower (but known, documented) success rate outside that range.
- **Multi-floor missions** — explicitly scoped OUT of the current build as a deliberate risk-management decision ("prioritise single-floor first; treat multi-floor as an extension"), but it's the clearest next expansion.
- **Feed Module 2 what it's asked for:** cover points for terrorists to use (currently zero exist, so no terrorist has ever fought from cover), a proper ``isExterior`` field on doors so exterior/corridor doors are pathable, and an authored/validated extraction (safe) zone instead of one invented ad hoc by the scene-building step.
- **Richer per-waypoint patrol data** (facing direction, dwell time, look targets) — right now patrol routes carry only positions.
- **Fix two known data bugs:** scenarios that can end up with two rooms both

labeled `hostage\_room`, and scenarios that fail validation but still get emitted and are playable anyway (an open decision: should the pipeline refuse to hand these back at all?).

- **Regenerate the evaluation report's missing figures** — the written report

currently references more tables/figures than are actually committed to the repo; this is flagged as a documentation debt, not a generation defect.

SAY THIS:

"The generator already supports more than the evaluator form currently exposes — difficulty, randomness, and hostage-risk level are all statistically validated to work, they're just not switched on in the live form yet. Beyond that, the clearest next step is feeding Module 2 what it's explicitly asked for — cover points and a working exterior-door flag — since we've measured exactly zero terrorists using cover today because that data simply doesn't exist yet on our side."

7. Inputs and outputs of each module (component), individually

The full pipeline, in order, each stage's exact input → output:

Stage	Class / method	Input	Output
0. Load config	`ScenarioConfigLoader.LoadFromFile`	`ScenarioConfig.json` on disk	validated `ScenarioConfig` object
1. Orchestrate	`ScenarioGenerator.Generate`	`ScenarioConfig`	final `ScenarioData` (calls everything below in order)
2. Layout	`LayoutGenerator.Generate`	`ScenarioConfig` + seeded RNG	`LayoutData` (rooms, doors, entry points, depth/type/zone metadata)
3. Entities	`EntityPlacer.Place`	`LayoutData` + `ScenarioConfig` + RNG	`EntityPlacementResult` (trainee/hostage/terrorist positions + spawn points)
4. Roles	`RoleAssigner.Assign`	`LayoutData` + entities + spawn points + config + RNG	`List<RoleAssignment>` (one role per terrorist: patrol / stationary guard / roaming guard / hostage guardian)
5. Navigation	`NavigationContextBuilder.Build`	layout + roles + entities + spawn points	per-NPC `NavigationContextEntry` (patrol routes, guard positions, facing directions)
6. Furniture	`FurniturePlacer.Place`	layout + entities + config + RNG (runs LAST so it can see every actor's final position)	`List<FurnitureData>`, written into each room
7. Assemble	inline in `ScenarioGenerator`	everything above	one `ScenarioData` object
8. Validate	`ScenarioValidator.Validate`	`ScenarioData`	`ValidationResult` (10 checks, pass/fail + warnings) — orchestrator retries on failure, up to 3 extra attempts with seed+1

Stage	Class / method	Input	Output
9. Export	<code>`ScenarioExporter.ExportToFile`</code>	<code>`ScenarioData`</code>	<code>`Scenario.json`</code> file on disk
10. Build scene (<i>downstream, Module 1→2 handoff</i>)	<code>`SceneBuilder.BuildScene`</code>	<code>`Scenario.json`</code>	actual Unity GameObjects — walls, doors, furniture meshes, spawned NPCs/hostage/trainee, baked NavMesh

What the INPUT config (``ScenarioConfig``) actually contains — three groups:

- ``missionStructure``: room count range (default 5–8), room size (small/medium/large), layout type (linear/branching/hub-and-spoke/loop), entry type (single/multiple).
- ``entityConfiguration``: hostage count (fixed at 1), terrorist count (1–8), placement strategy (clustered/dispersed/front-loaded/deep), hostage risk level (low/medium/high — controls how close terrorists must be to the hostage).
- ``executionControls``: difficulty (1–5), randomness (low/medium/high), seed (for reproducibility), time limit, a free-text label.

What the OUTPUT (``Scenario.json`` / ``ScenarioData``) actually contains: scenario ID, mission type, the full ``LayoutData`` (rooms with positions/sizes/ doors/furniture), ``SpawnPointData`` for every actor, the entity list, the role assignment list, the navigation context dictionary, and a ``ConfigurationMetadata`` block that embeds the *entire original input config* plus the actual seed used, generator version, and the validation result — this last part exists specifically so anyone downstream (or an evaluator) can trace exactly what settings produced this exact scenario.

SAY THIS:

"Each stage has a clean, single-direction data contract — layout goes in, entity placement comes out; entities and layout go in, roles come out; roles and layout go in, navigation data comes out. Nothing loops back. And the final output always carries its own complete input config embedded inside it, specifically so a scenario is traceable — you can always answer 'what settings produced this exact mission.'"

8. Where/how did you find this technique — literature sourcing

Simple version: We wrote a formal literature review first (22 sources), then designed the algorithm to match what the evidence supported, and wrote a document that maps every single design decision to the specific paper that justifies it.

The clearest, most citable examples an evaluator will respect:

- **Why not WFC?** Karth & Smith, *"WaveFunctionCollapse is Constraint Solving in the Wild"* (FDG 2017) — proved WFC can't guarantee reachability without expensive backtracking.
- **Why doors as room attributes, not separate objects?** Smith & Whitehead,

"Analyzing the Expressive Range of a Level Generator" (PCG Workshop @ FDG, 2010).

- **Why BFS-depth-based role assignment?** Dahl & Rinde's 2008 Chalmers

master's thesis on procedural indoor generation (Space Syntax precedent), and Horswill & Foged, "Fast Procedural Level Population with Playability Constraints" (AIIDE 2012) for the "resistor network" depth/flow analogy.

- **Why hierarchical (trainee → hostage → terrorist) placement order?**

Raistrick et al., "Infinigen Indoors" (CVPR 2024) — a real, large-scale, peer-reviewed procedural indoor scene generator.

- **Why difficulty-aware danger positioning (behind doors, blind corners)?**

Zechner et al., "Enhancing Operational Police Training in High Stress Situations with VR" (2023) — a real EU-funded (Horizon 2020 SHOTPROS) study with 96–234 real participants, measuring which staged events cause the most stress.

- ****Why did we generate a single validated scenario per request instead of**

exploring the whole possibility space?* Hullett & Mateas, "Scenario Generation for Emergency Rescue Training Games"* (FDG 2009) — their system hit memory exhaustion trying to exhaustively enumerate scenarios, so we took that as a direct warning and designed around it.

- **Why does the output embed its own input config?** Cook, "Procedural

Generation and Information Games"* (IEEE CoG 2020) — argues that if an evaluator interprets a generated feature as meaningful but it was actually arbitrary, the training evidence itself becomes unreliable. That's the traceability requirement.

SAY THIS:

"Every major design decision in Module 1 traces back to a specific paper — we kept a literature review with 22 sources and wrote three algorithm design documents that each end with a table mapping every decision to its citation. It wasn't 'we picked a technique that seemed reasonable' — for example, doors being attributes of rooms rather than independent objects comes directly from a 2010 finding by Smith and Whitehead that the alternative makes connectivity significantly harder to guarantee."

9. What happens when different conditions (variables) change, and what do the results look like?

This is essentially Experiments B, C, and D, retold as "if you turn this dial, here's what happens":

- **Change `layoutType`** (linear/branching/hub-and-spoke/loop) → ****huge,**

certain effect** on graph connectivity and cyclicity ($\epsilon^2=1.000$, the maximum possible effect size) and on max depth ($\epsilon^2=0.890$) and patrol coverage ($\epsilon^2=0.758$). This is the single strongest lever in the whole system.

- **Change `roomCount`** → certain effect on actual room count (obviously,

$\epsilon^2=1.000$) but also on graph diameter ($\epsilon^2=0.734$) and total furniture ($\epsilon^2=0.611$) — more rooms means a bigger building and proportionally more furniture.

- **Change `roomSize`** → strong effect on furniture-per-room ($\epsilon^2=0.878$) and

average entity distance from the entrance ($\epsilon^2=0.689$) — bigger rooms hold more furniture and let entities spread out further.

- **Change `placementStrategy`** → moderate-to-large effect on how spatially clustered entities are ($\epsilon^2=0.327-0.420$).

- **Change `terroristCount`** → certain effect on patrol count ($\epsilon^2=1.000$) and large effect on how many end up as roaming guards ($\epsilon^2=0.801$).

- **Change `entryType`** (single vs multiple) → **only** affects entry-point

count. It does NOT change interior room connectivity or door-open rates at all — this is the one parameter we found with a genuinely null downstream effect, and we report it as a documented limitation rather than hiding it.

- **Change `randomnessLevel`** (low/medium/high) → almost nothing changes.

Room-graph topology (connectivity, cyclicity, room count) is completely constant across all three levels. Only one very minor metric (`entityDepthVariance`) showed any drift, and it wasn't statistically significant ($p=0.061$).

Finding, reported honestly: at the currently-fixed 4-room baseline, structural topology is essentially deterministic regardless of the randomness dial.

- **Change `difficultyLevel`** (1→5) → real, meaningful changes: patrol routes

vanish entirely at difficulty 5 (patrol count and coverage both hit exactly 0), stationary guards appear where they weren't before (0.00 → 0.66 average count going from difficulty 1 to 3), and the minimum distance kept between hostage and terrorists climbs steadily (3.00m → 3.51m → 3.93m) — meaning higher difficulty pushes terrorists into more spread-out, guard-heavy, patrol-free configurations. Interestingly, WHERE entities sit relative to the entrance barely moves (not significant) — difficulty changes *who does what*, not *how far away things are*.

SAY THIS:

"layoutType and terroristCount are the strongest levers we found — both hit the maximum possible effect size on their primary metric. Difficulty doesn't just reposition things, it changes the entire behavioural mix: at maximum difficulty, patrols disappear completely and get replaced by static guards holding more distance from the hostage. And we found one honest null result — the 'multiple entries' setting only adds entry points, it doesn't restructure the building's interior — which we report as a limitation rather than papering over."

10. "Why did you do it that way? How did you think it would work? What results did you find?"

This question wants the **reasoning chain**: prediction → design → actual outcome. Three strong worked examples:

Example 1 — constructive generation instead of WFC.

Why: the literature (Karth & Smith 2017) showed WFC struggles to guarantee a level is fully walkable. *What we predicted:* a hand-built constructive generator with an explicit, separate reachability check would produce reliably walkable missions without needing expensive backtracking. *What we found:* Experiment E — **100% validation pass rate** within the evaluator-selectable range (43/43 scenarios), and even across the FULL extended range the pipeline supports, only 2 of 10 validation checks ever failed (`EntityBounds` and `EntityOverlap` — both entity-placement issues, not reachability failures). Reachability itself was never the failure category — directly confirming the design bet paid off.

Example 2 — BFS-depth-based role assignment instead of random roles.

Why: Space Syntax theory (Dahl & Rinde 2008) says spatial depth correlates with privacy/threat level in real buildings, and Cook (2020) argues randomly arbitrary placement undermines training value because evaluators can't trust what they're seeing means anything. *What we predicted:* if roles are derived from room depth rather than randomly rolled, difficulty settings should produce *coherent, interpretable* changes in NPC behaviour mix, not just noise. *What we found:* Experiment D confirmed this precisely — difficulty 5 doesn't just "add more guards," it specifically and completely eliminates patrols ($\epsilon^2=0.760$, a large effect) while pushing minimum hostage-terrorist distance up in a smooth, monotonic way (3.00→3.51→3.93m) — a coherent tactical story, not random noise.

Example 3 — embedding the full input config inside the output.

Why: Cook (2020) — if a training evidence trail can't be traced back to what actually produced it, the evidence is unreliable. *What we predicted:* every scenario should be fully reconstructable from its own output file alone. *What we found:* this is exactly why `ConfigurationMetadata` carries the complete original `ScenarioConfig` plus the exact seed used — we verified this directly by re-running the same seed and confirming identical output (this is literally the mechanism Experiment A's whole methodology depends on being true).

SAY THIS:

"Take role assignment as the clearest example. We didn't roll dice for NPC roles — we predicted, based on Space Syntax theory, that deriving roles from how spatially deep a room is would produce a coherent difficulty curve rather than noise. Experiment D confirmed that specifically: difficulty five doesn't just add enemies, it eliminates patrols entirely and smoothly increases hostage-terrorist spacing as difficulty rises — that's a predictable, coherent tactical pattern, which is exactly what the theory predicted and exactly what we measured."

11. Code snippets for calculation parts (have these ready to show)

Seeding / reproducibility (`ScenarioGenerator.cs:108-118`):

```
int resolvedSeed = config.executionControls.seed ?? Environment.TickCount;
int seedInUse = resolvedSeed;
for (int attempt = 0; attempt <= MaxLayoutRetries; attempt++)
{
    System.Random rng = new System.Random(seedInUse);
    LayoutData layout = _layoutGenerator.Generate(config, rng);
    ...
}
```

Explain: one seed drives the ENTIRE pipeline (every generator receives the same `rng`), so the same seed always reproduces the exact same mission — this is what makes Experiment A's "regenerate the same config 100 times" study meaningful.

Hostage-risk distance constraint (`EntityPlacer.cs:349`, paraphrased shape):

```
// High risk:    [0,                1.5 × roomWidth]
// Medium risk:  [0.5×roomWidth, 3.0 × roomWidth]
// Low risk:     [2.0×roomWidth, ∞]
```

Explain: this is the exact formula controlling how close/far terrorists must be placed from the hostage — directly implements the distance-constraint idea from Cheng, Han & Fei (2020)'s WFC-extension paper.

Role quota decision (`RoleAssigner.cs:196`, concept):

```
var quotas = GetRoleQuotas(remainingTerroristCount, difficultyLevel);
// returns (patrolCount, roamingCount, guardCount) from a hard-coded
// difficulty-bracketed lookup table
```

Explain: this is the table Experiment D's difficulty results measure — low difficulty biases toward patrol, high difficulty biases toward stationary guard.

Reachability check (`LayoutGenerator.cs:839`, concept):

```
// BFS from the entry room across connectedRoomIds
// throws (triggering a retry with seed+1) if visited.Count != rooms.Count
```

Explain: this is what guarantees every generated mission is fully walkable — the thing WFC couldn't reliably guarantee, per Karth & Smith (2017).

Validation — 10 independent checks, always all run (`ScenarioValidator.cs:68`):

```
// CheckRoomBounds, CheckEntityBounds, CheckDoorConsistency,
// CheckEntityMetadata, CheckReachability, CheckHostagePath,
// CheckEntityOverlap, CheckNavigationContext, CheckReferentialIntegrity,
// CheckRoleAssignmentCompleteness
// passed = (checksPassed == 10); never throws, always returns a result
```

Explain: this is exactly what Experiment E's pass/fail numbers are counting — 10 checks per scenario, and a scenario only "passes" if literally all 10 do.

Metric extraction (the thing every experiment's numbers come from)

(`ScenarioMetrics.cs`, concept):

```
// Extract() reads the FINAL Scenario.json only – never generator internals –
// so any batch of exported scenarios can be re-measured independently,
// producing one row of 39 metrics per scenario for the statistics to run on.
```

SAY THIS, if they push on "show me the code":

"Every number I've quoted traces back to a specific function — the seed propagation you're seeing here is exactly what makes our variability study valid, because every generator stage shares the same `System.Random` instance, threaded through as a parameter, never re-seeded internally."

Quick-reference number sheet (for last-minute review)

- **22** literature sources reviewed.
- **4** research questions (RQ1–RQ4), one per experiment group.
- **10** independent validation checks per scenario, all always run.
- **39** metrics extracted per scenario.
- **7** pairwise diversity measures.
- **100%** validation pass rate within the evaluator-selectable range (43/43).
- **83.2%** pass rate across the FULL technical range (416/500).
- **10.06m** average entity-position distance between two repeat-generated scenarios (medium randomness, same config).
- **0%** CV (zero variation) in room-graph topology at fixed settings.
- **1.000** (maximum) effect size for layoutType→connectivity and terroristCount→patrolCount — the two strongest levers found.
- **3.00m → 3.93m** — minimum hostage-terrorist distance climbing from difficulty 1 to difficulty 5.
- **1.8 seconds** to generate the entire 2000-scenario evaluation dataset, 0 exceptions.